

DYNAMIT: A T-Pot-based Adaptive Honeynet System Using K-Medoids Clustering for Dynamic Honeypot Profiling

Zaki Ananda

*Department of Electrical Engineering
Universitas Indonesia
Depok, Indonesia*

Yan Maraden, S.T., M.T., M.Sc.

*Department of Electrical Engineering
Universitas Indonesia
Depok, Indonesia*

Prof. Dr. Ir. Riri Fitri Sari, M.M., MSc.

*Department of Electrical Engineering
Universitas Indonesia
Depok, Indonesia*

I Gde Dharma Nugraha, S.T., M.T., Ph.D

*Department of Electrical Engineering
Universitas Indonesia
Depok, Indonesia*

Abstract—As the internet continues to grow, the number of cyberattacks also increases. Many organizations have built cybersecurity systems in order to address that problem. One component of such systems is the honeypot, which is designed to collect attack data by luring and trapping attackers. However, most existing honeypot implementations can be easily detected by attackers due to their manual deployment. To overcome this limitation, a system called DYNAMIT was developed, capable of automatically deploying honeypots and blending them into production networks. To ease its development challenge, T-Pot is used as DYNAMIT's development base. To generate honeypot profiles, DYNAMIT first will perform network clustering using K-Medoids algorithm in order to gain information regarding network characteristics. It will then generate honeypot profiles based on the result of said clustering. DYNAMIT then utilizes Docker and MACVLAN to simulate honeypot profiles, including their own IP address, MAC address, and list of services, all while only utilizing one physical host. While implementing K-Medoids to perform network clustering, an optimal parameter was found using case when total host is 100. When tested, it was capable of accurately capturing the correct number of clusters, matching the intended generated clustering parameter with a relative error of 18.75% when total host is 40, and 6.625% when total host is 1000. Resource usage evaluation revealed that while idle, DYNAMIT consumes 5% CPU usage, approximately 5.3 GB of RAM to run its core components, and an additional 180–250 MB of RAM for each honeypot profile it operates. Boot time evaluation indicated that when honeypot profile generation is performed, DYNAMIT requires 231–249 seconds before the system becomes operational. Without this process, DYNAMIT only requires 86–126 seconds.

Index Terms—cybersecurity, cyberattacks, Docker, honeypot, K-Medoids, network clustering, T-Pot.

I. INTRODUCTION

As internet adoption continues to grow rapidly, so does the number and sophistication of cyberattack cases [1]. This ever-expanding threat has led many organizations to invest heavily in cybersecurity systems [2]. One such defensive mechanism is the honeypot, a vulnerable host designed to lure and trap attackers in order to collect valuable attack data [3]. Such data

can then be used for further analysis and threat intelligence [4].

However, setting up honeypots has its own challenges. One of the main difficulties is how to deceive attackers convincingly enough such that they would engage with the trap, rather than identifying and avoiding it [5]. Skilled adversaries are often capable of identifying whether a host is actually a honeypot or not, such as by performing a comparison with a list of known honeypot signatures, or by checking inconsistencies within the network topology [6]. Furthermore, many honeypots today are either configured manually, which is labor-intensive and difficult to scale, or they rely on default settings, which is trivial for attackers to fingerprint and identify [7].

To address the gap between scalability and deception capability, this research is focused on the development of a new system called DYNAMIT, short for Dynamic T-Pot. DYNAMIT is a dynamic honeypot platform designed to automatically adapt to its surrounding network environment, allowing it to better mimic legitimate systems and blend seamlessly into real infrastructure, thereby increasing its chances of deceiving even sophisticated attackers.

II. RELATED WORKS

Dynamic honeypot is a type of honeypot capable of self-configuration, adapting to network behavior or condition. They are often contrasted with static honeypot, a more conventional implementation that lacks such flexibility. Compared to static honeypots, dynamic honeypots are generally preferred due to their scalability, lower maintenance cost, and greater adaptability [8] [7].

Zhang et al. [9] proposed a dynamic honeypot design based on network scanning process. This design utilizes Nmap and P0f to perform the scans. The results are then used to cluster hosts using K-Means clustering, where the distance between each hosts is calculated with Manhattan distance, comparing

all features from the network scans. The honeypots are then set up using Honeyd, using the median profile of each cluster, applying a random offset to ensure profile diversity.

Fraunholz et al. [10] proposed a similar design to Zhang et al.'s. It relies solely on Nmap for its network scanning process. Similar to Zhang's approach, host clustering is performed using K-Means, with Manhattan distance to measure similarity between hosts. To approximate the global optimum more effectively, the author applied a heuristic method, and concluded that 15 clustering iterations is needed. Another heuristic is used to find the optimal number of cluster, based on a threshold derived from the variance scores of the clustering results. The honeypots are then deployed using Honeyd. For each honeypot profile, the base profile is selected from the biggest cluster, whose effective size is subsequently reduced by half to ensures better profile distribution across clusters.

Bartwal et al. [11] proposed a different design, adapting to attack traffic instead of network scan results. The system monitors for traffic directed at unused IP addresses, a typical sign that an attacker is probing the network. It then predicts the next likely target IP address, and deploys a honeypot profile at that address, with a deployment time of 6 seconds. To deploy the profile, the system uses either a Docker container running a real service, or a honeypot container emulating a service. Once deployed, the system monitors the attacker's behavior on the honeypot, and dynamically orchestrate additional honeypots based on the observed attack types. The result is a system that continuously diverts attackers into a network of deceptive targets, effectively wasting their time and resources.

Song et al. [12] proposed another approach to dynamic honeypots, focusing on bait-and-switch tactic. The system would monitor for malicious traffic directed to a real host. Upon detection, the system would then deploy a virtual machine as a honeypot. The malicious traffic is then redirected to the honeypot using OpenDaylight SDN controller, effectively isolating the attacker from the actual system while maintaining the illusion of a legitimate target.

III. DYNAMIT IMPLEMENTATION

A. Design Requirements

DYNAMIT is a dynamic honeypot platform, the development of which is presented in this paper. The name is derived from "Dynamic T-Pot", reflecting T-Pot as its development base.

Dynamic honeypots provide two primary advantages over static implementations: the ability to automatically deploy honeypot instances, and the capacity to adapt to specific network environments [7]. Because of that, DYNAMIT is designed to fulfill such requirements:

- 1) DYNAMIT must be able to perform automated honeypot deployment. To do so, DYNAMIT builds upon T-Pot, an existing open source platform capable to automatically deploy a preconfigured set of honeypot containers. In addition, T-Pot also facilitates centralized logging process for every deployed honeypots by utilizing ELK Stack (Elasticsearch, Logstash, Kibana).

- 2) DYNAMIT must be capable of analyzing and learning the characteristics of its network environment, in order to deploy honeypots that closely resemble legitimate hosts. A large portion of this paper is focused on the design, implementation, and evaluation of this capability.

B. System Architecture

To ensure proper isolation between management access and attacker interaction, DYNAMIT must be connected to two different networks: a management network and a production network. Because of that, DYNAMIT host must have two differing network interface: honeynet interface connected to the production network, and management interface connected to the management network. System will expose honeypot profiles to the production network, in order to lure attacker who have successfully infiltrated the network. On the other side, system will also expose management services to the management network, allowing cybersecurity team to monitor and maintain DYNAMIT securely through a dedicated web interface. The topology of DYNAMIT deployment is illustrated in Fig. 1.

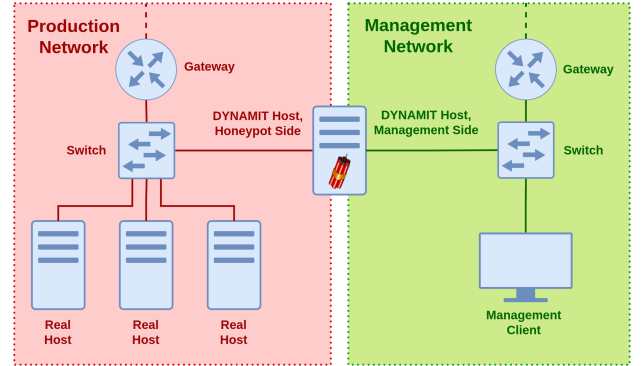


Fig. 1. DYNAMIT deployment network topology

Before deploying honeypot profiles, DYNAMIT must first perform a network scan to gather information about the production network's characteristics and its surrounding hosts. From this scan, DYNAMIT will collect various data such as IP address, MAC address, open ports, services exposed by each hosts within the network. The scan result is then processed using a clustering algorithm. The result of such clustering is then used generate certain amount of honeypot profiles. Each of those honeypot profiles will emulate a physical host, with their own IP address, MAC address, and set of exposed ports and services. DYNAMIT will then deploy those profiles, using MACVLAN to emulate the networking stack of a honeypot profile, and containerized honeypot to emulate the service stack. From the attacker's perspective, these profiles are indistinguishable from other legitimate, physical hosts. Additionally, each honeypot container is configured to forward logging data to DYNAMIT for monitoring and observability purpose. These operational flows are illustrated in Fig. 2.

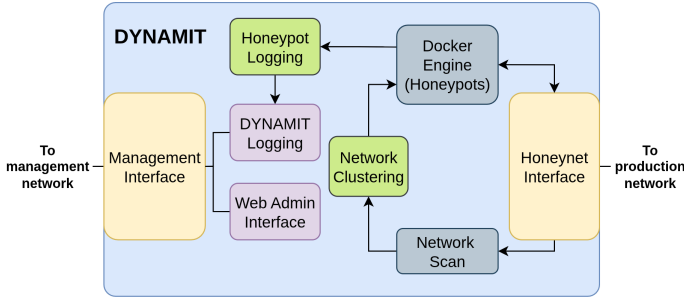


Fig. 2. Flow of DYNAMIT operation

C. Orchestration Flow

DYNAMIT orchestration flow begins with a systemd service named `dynamit.service` that runs automatically at every boot. The purpose of this service is to launch a startup script named `dynamit-start.sh`, which serves as the entry point to start the platform. The script first loads runtime parameter from a `.env` file, and subsequently executes a container named `dynamit-builder`. This container would perform the honeypot profile creation process, from network scanning to profile deployment. The output of such process is a Docker Compose file named `dynamit-run.yaml`, which defines the core component for DYNAMIT along with the generated honeypot profile. Next, the script removes the IP address assigned to the honeynet interface in order to isolate DYNAMIT host's networking stack from the production network. This ensures that only the honeypot profiles are exposed to the production network. Finally, the script runs the Docker Compose file, thereby launching both the DYNAMIT core components and the configured honeypot profiles. These flows are illustrated in Fig. 3.

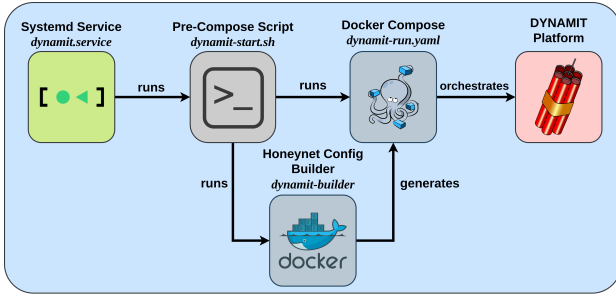


Fig. 3. Flow of DYNAMIT orchestration

D. Network Scanning

Network scanning is done using Nmap, leveraging `python-libnmap` for ease of scan result processing. Nmap performs a scan according to the production network subnet, as specified in the configuration file, with the following command and arguments:

```
nmap -n -sS -p- -sV -T4 <prod_ip>
```

- 1) `-n` to ignore reverse DNS resolution, an irrelevant process in our local network scan
- 2) `-sS` to perform standard TCP port scan
- 3) `-p-` to perform scan on all TCP port (1-65535)
- 4) `-sV` to perform service fingerprinting, in order to learn what type of service is running on scanned host
- 5) `-T4` to set the scanning speed to aggressive, recommended for LAN scanning

From the scan result, the system obtains IP address, MAC address, exposed ports, and list of services for all hosts present inside production network.

E. Network Clustering

Network clustering is then performed from the scanning result. Two clustering features are selected: OUI and list of services. OUI or the first 3 bytes of MAC address identifies host similarity from hardware perspective, as two hosts with the same brand of network interface will have the same OUI string. On the other hand, list of services identifies host similarity from software perspective, as two host with the same set of service is likely to have same intended usage.

To quantify host similarity based on OUI feature, Manhattan distance is used, with the following equation, where $d_{OUI}(a, b)$ is the OUI distance between host a and host b , and OUI_x is the OUI value of host x

$$d_{OUI}(a, b) = \begin{cases} 0, & \text{if } OUI_a = OUI_b \\ 1, & \text{if } OUI_a \neq OUI_b \end{cases} \quad (1)$$

With such metric, a pair of hosts with the same value of OUI will have OUI distance of 0, or nearest distance. Conversely, a pair of host with a different value of OUI will have OUI distance of 1, or furthest distance.

On the other hand, to quantify host similarity based on list of services, Jaccard distance is used, with the following equation, $d_{svc}(a, b)$ is the service distance between host a and host b , and svc_x is the set of services present in host x

$$d_{svc}(a, b) = \begin{cases} 1 - \frac{|svc_a \cap svc_b|}{|svc_a \cup svc_b|}, & \text{if } |svc_a \cup svc_b| \neq 0 \\ 0, & \text{if } |svc_a \cup svc_b| = 0 \end{cases} \quad (2)$$

With such metric, a pair of host with high amount of service intersection will have service distance near 0, or short distance. Conversely, a low amount of service intersection will have service distance near 1, or long distance.

Combining the two distance, the total distance $d(a, b)$ for any pair of hosts a and b is then calculated as the average of their OUI distance and their service distance.

$$d(a, b) = \frac{d_{OUI}(a, b) + d_{svc}(a, b)}{2} \quad (3)$$

The aforementioned distance metric is then calculated for every pair of hosts present inside the production network, forming a dissimilarity matrix of $n \times n$ size, where n is the amount of host inside the network. The next step then is to perform clustering based on the calculated dissimilarity matrix.

Several clustering algorithms were considered, but ultimately, K-Medoids was chosen for its similarity with K-Means used by implementations similar to this paper [9] [10], its flexibility in terms of distance metric, its capability to represent each cluster using medoids, and its optimisation-based approach [13]. To perform K-Medoids, a python library named `kmedoids` [14] is used for its up-to-date implementation. Specifically, DYNAMIT uses `kmedoids` implementation of FasterPAM, as it is recommended for use in multitude of applications.

One of the biggest drawback of using K-Medoids is its incapability to find the optimal number of cluster, or k_c , as the algorithm requires the number of cluster, or k , to be known before the clustering process itself is performed. As workaround, DYNAMIT will perform multiple iterations of clustering process, from $k = 1$ to $k = k_{max}$, whose value is set inside the configuration file. The clustering scores from those iterations are then evaluated, to find an iteration that fulfills certain threshold. Discussion regarding this threshold will be continued in Section 4.

F. Profile Generation

The clustering result is then used as basis for honeypot profile generation. Each of those profiles will emulate a physical host, with their own IP address, MAC address, exposed ports and services. To generate those profile, an approach similar to Fraunholz [10] is used:

- 1) The amount of honeypot profile it needs to generate are calculated with the following equation

$$\text{profile_count} = \left\lceil \frac{\text{HPOT_HOST_RATIO} \times \text{host_count}}{\text{host_count}} \right\rceil \quad (4)$$

where the ratio is defined in the configuration file.

- 2) Next, for every profile it needs to generate, the system will randomly choose an existing host from the largest cluster. Afterwards, the effective size of the largest cluster is reduced to half, to ensure better profile distribution across clusters.
- 3) From those base profile, the IP address and last 3 byte of MAC address is mutated, as their value must be unique inside the network. On the other hand, the first 3 byte of MAC address (OUI) and the list of services are kept exactly the same as the base profile

G. Profile Deployment

Following the honeypot profile generation, DYNAMIT then constructs a customized Docker Compose file in order to emulate these profiles. Aligning closely with T-Pot's security practices, all containers specified in the Compose file must be deployed as read-only, non-root, and with whitelisted kernel capabilities, in order to minimize the risk of container escape or privilege escalation. To construct the Compose file, DYNAMIT begins with a predefined template that only includes the core system components. Then, for each generated honeypot

profile, the system adds an NGINX container along with a set of honeypot service containers, according to the service specification of the profile. The NGINX container is assigned a unique IP and MAC address, in accordance with the honeypot profile, and is connected to the MACVLAN network to make it externally reachable. For the service containers, DYNAMIT currently supports emulation of 5 services, selected based on surveys about commonly targeted protocols [15]:

- 1) HTTP (Snare-Tanner)
- 2) SSH (Cowrie)
- 3) SMB, RDP, and VNC (all via qeeqbox-honeypots)

If a profile specifies an unsupported service, DYNAMIT leaves the corresponding port open but does not emulate it. Each service containers are then connected to an internal Docker network, enabling inter-container communication with the corresponding NGINX instance. NGINX is then configured to forward incoming traffic on specific ports to the appropriate honeypot service containers using TCP stream. This creates the illusion that a single host is running multiple services behind its designated IP and MAC. Additionally, Logstash is configured to collect logs from the honeypot service containers, enabling centralized monitoring and analysis. However, this architecture introduces a limitation: only the NGINX container has visibility into the attacker's source IP address. Consequently, the cybersecurity team must perform log correlation between NGINX and honeypot containers to reconstruct attacker activity.

Once deployed, DYNAMIT is capable of emulating multiple physical hosts, each with a unique IP address, MAC address, and set of exposed services, all within a single physical host. As illustrated in Fig. 4, an attacker interacting with the system perceives these honeypot profiles as distinct physical hosts, each running its own services. The DYNAMIT host itself is rendered invisible from the attacker's perspective, as the honeynet interface is deliberately left without an assigned IP address, and ARP packets directed to the host are blocked to prevent identification.

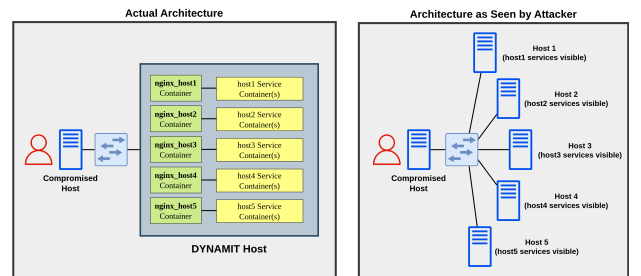


Fig. 4. Actual architecture vs attacker's perspective

Fig. 5 depicts the packet flow during an attack on a honeypot profile. When a malicious packet is sent to a specific port, it is first received by the NGINX container. NGINX then forwards the packet to the appropriate honeypot service container. From the perspective of the honeypot container, the attack appears to originate from the NGINX container. Upon generating a

response, the honeypot container sends it back to NGINX, which in turn forwards the response to the actual attacker. This design creates the illusion of a single physical host running multiple services, all emulated with just Docker containers and MACVLAN networking. However, as noted before, a significant limitation of this design is the lack of attacker's IP visibility from the honeypot container perspective, as all traffic appears to originate from the NGINX container.

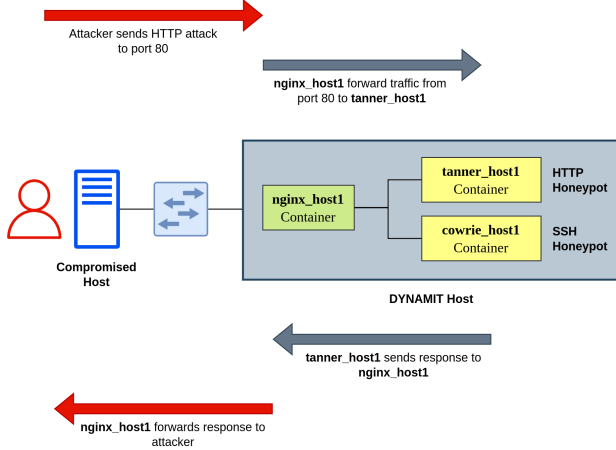


Fig. 5. DYNAMIT attack flow

IV. EVALUATION

A. Hypothesis for Optimal K-Medoids Parameter

To find the optimal number of cluster using K-Medoids, a threshold for the clustering score must be used. In order to find an optimal threshold, an observation is conducted on a randomly generated dataset with a total host of 100

To identify the optimal number of clusters using the K-Medoids algorithm, this study hypothesizes that a threshold derived from clustering scores is required. To determine the optimal threshold, an observational study was conducted by generating a synthetic dataset, which was then processed using a clustering algorithm. This dataset consists of a number of hosts, each characterized by two main data columns: the OUI and a list of services. The dataset is designed with an implicit clustering structure, with h_c amount of hosts per cluster, k_c amount of clusters inside dataset, based on the values of its two main attributes.

Once the dataset was generated, clustering was performed using the K-Medoids algorithm, with the number of clusters from $k = 1$ to $k = k_c + 10$. The results of these clustering processes were then evaluated by calculating their clustering scores, consisting of loss value L and silhouette scores S . These values were then compared, and clustering results that met the threshold criteria are determined to be the optimal number of clusters.

To formulate a hypothesis regarding the optimal K-Medoids parameters, observations were carried out under the condition of a network consisting of 100 total hosts ($h_c \times k_c = 100$). The study examined three different (h_c, k_c) pairs: (5, 20), (10, 10),

and (20, 5), and evaluates their both of their clustering scores. The clustering scores for those cases are plotted in Fig 6.

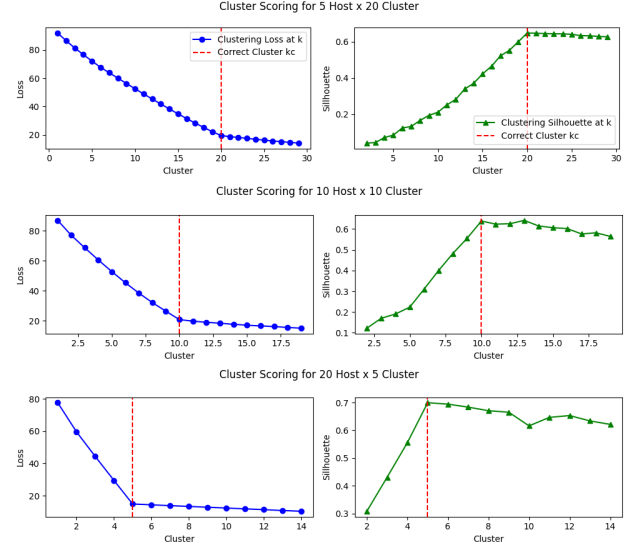


Fig. 6. Clustering scores compared to k for three pair of (h_c, k_c)

Based on our observations, we found that as the number of clusters k approaches the optimal value k_c , both the loss L and silhouette scores S begin to plateau, or typically referred in machine learning world as the formation of an elbow point. We hypothesize that the optimal number of clusters can be identified by locating a value of k , where this elbow point is evident in both the loss and silhouette metrics. To systematically detect this point, we further hypothesize that it can be determined by analyzing the difference between clustering scores for successive values of k . Specifically, we define a difference threshold based on the observed change in score between $k = k_c - 1$ with $k = k_c$ (pre-plateau), and between $k = k_c$ with $k = k_c + 1$ (post-plateau). These score differences are summarized in Table I.

TABLE I
COMPARISON BETWEEN PRE-PLATEAU AND POST-PLATEAU DIFFERENCE FOR LOSS AND SILHOUETTE

h_c	k_c	Loss Difference		Silhouette Difference	
		$L_{k_c-1} - L_{k_c}$	$L_{k_c} - L_{k_c+1}$	$S_{k_c} - S_{k_c-1}$	$S_{k_c+1} - S_{k_c}$
5	20	2.34	1.03	0.042	0.007
10	10	6.2	1.0	0.12	-0.028
20	5	14.8	0.5	0.17	-0.013

From these observations, we define the threshold as the average of the minimum pre-plateau score difference and the maximum post-plateau score difference, calculated separately for the loss and silhouette scores. The resulting threshold formulation for the loss score is presented in (5), and for the silhouette score in (6).

$$\begin{aligned}
L_{k_c} - L_{k_c+1} &\leq \frac{(\min(L_{k_c-1} - L_{k_c}) + \max(L_{k_c} - L_{k_c+1}))}{2} \\
&\leq \frac{2.34 + 1.03}{2} \\
&\leq 1.685
\end{aligned}$$

$$L_{k_c} - L_{k_c+1} \leq 1.8 \quad (\text{Added tolerance}) \quad (5)$$

$$\begin{aligned}
S_{k_c+1} - S_{k_c} &\leq \frac{(\min(S_{k_c} - S_{k_c-1}) + \max(S_{k_c+1} - S_{k_c}))}{2} \\
&\leq \frac{0.042 + 0.007}{2} \\
&\leq 0.245
\end{aligned}$$

$$S_{k_c+1} - S_{k_c} \leq 0.03 \quad (\text{Added tolerance}) \quad (6)$$

Finally, we combine both threshold formulations into a unified selection criterion to determine the optimal number of clusters k_c , ensuring that the elbow point is validated by both metrics.

$$\begin{aligned}
k_c = \min \left\{ k \in \mathbb{N} \mid L_k - L_{k+1} \leq 1.8 \right. \\
\left. \wedge S_{k+1} - S_k \leq 0.03 \right\} \quad (7)
\end{aligned}$$

B. K-Medoids Parameter Evaluation

With the criteria for determining the optimal number of clusters formally defined, we proceed to evaluate its accuracy under varying conditions. This evaluation is conducted across two distinct scenarios: one involving a total of 40 hosts and another involving 1,000 hosts. For each scenario, we assess the performance of the proposed parameter across 4 different cases, each characterized by a uniquely generated dataset, a specified number of hosts per cluster (h_c), and a known optimal number of clusters (k_c). During the evaluation, each dataset is clustered multiple times using different values of k , and the parameter attempts to predict the optimal number of clusters (predicted k_c), which is then compared against the ground-truth optimal value (actual k_c). This approach allows us to measure the parameter's predictive accuracy and robustness across datasets of varying scale and structure.

The evaluation results for the first scenario, involving a total of 40 hosts, are visualized in Fig. 7 and summarized in Table II.

TABLE II
COMPARISON BETWEEN GROUND TRUTH FOR k_c AND k_c AS PREDICTED BY THRESHOLD FOR SCENARIO WHERE TOTAL HOSTS IS 40

h_c	Actual k_c	Predicted k_c	Relative error of k_c
2	20	5	75%
5	8	8	0%
8	5	5	0%
20	2	2	0%
Average			18.75%

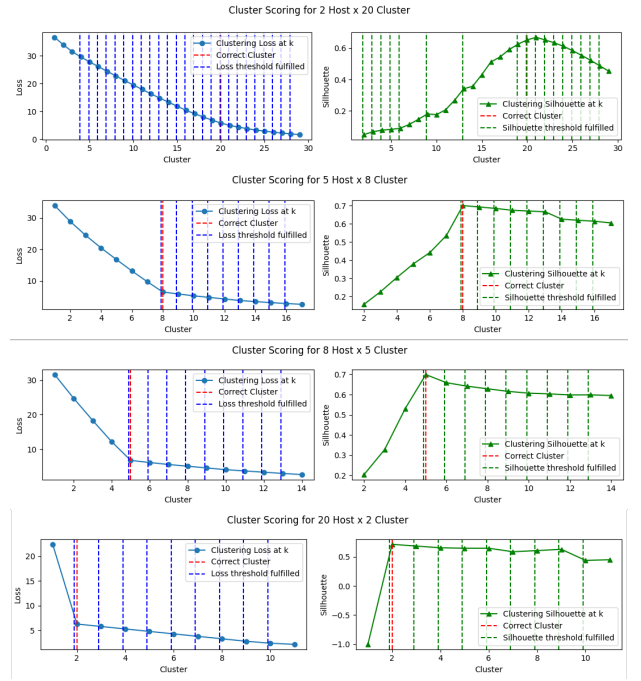


Fig. 7. Clustering scores and k that fulfills loss and silhouette threshold, in various cases where total hosts is 40

In the first case, with $h_c = 2$ and $k_c = 20$, the parameter failed to accurately predict the optimal number of clusters, instead producing a prediction of $k = 5$. As shown in the corresponding plot, the criteria were fulfilled at many values of k well before the actual k_c , for both the loss and silhouette scores. This indicates that the parameter was unable to correctly detect the elbow point, resulting in an incorrect prediction for that dataset. In contrast, for the remaining three cases, the parameter successfully predicted the optimal number of clusters. The corresponding graphs show that the selection criteria were met only at $k = k_c$ and for every k value afterwards. These results demonstrate the parameter's effectiveness in capturing the true elbow point and, consequently, its success in predicting the correct number of clusters for those specific datasets.

The evaluation results for the second scenario, involving a total of 1,000 hosts, are presented in Fig. 8 and summarized in Table III.

TABLE III
COMPARISON BETWEEN GROUND TRUTH FOR k_c AND k_c AS PREDICTED BY THRESHOLD FOR SCENARIO WHERE TOTAL HOSTS IS 1000

h_c	Actual k_c	Predicted k_c	Relative error of k_c
10	100	100	0%
25	40	41	2.5%
40	25	26	4%
100	10	12	20%
Average			6.625%

While the parameter accurately predicted the optimal number of clusters in the first case, it slightly missed the optimal value by one or two clusters in the remaining cases. Upon

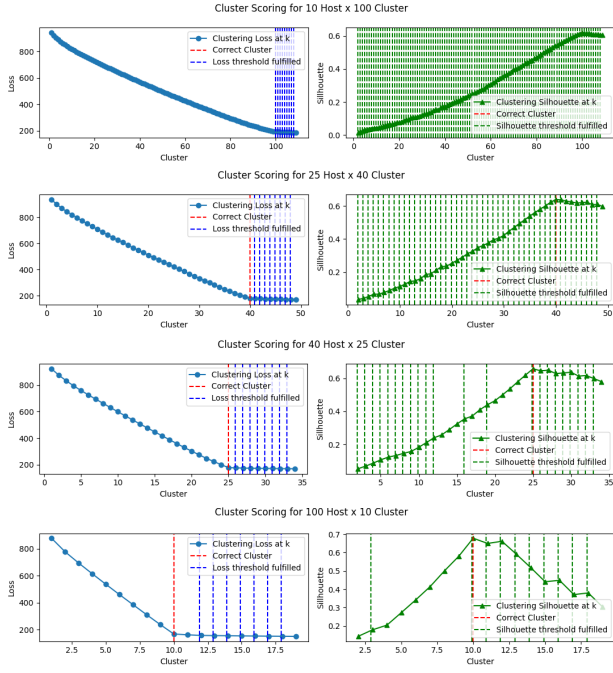


Fig. 8. Clustering scores and k that fulfills loss and silhouette threshold, in various cases where total hosts is 1000

examining the plots, it is evident that the parameter consistently captured the elbow point for the loss score across all cases. However, it failed to identify a clear elbow point for the silhouette score in three out of the four cases. Despite this, the final predictions remained largely unaffected, due to the fact that the parameter's selection criterion requires simultaneous satisfaction of both for loss and silhouette threshold.

C. Resource Usage Evaluation

To clearly define the system requirements for deploying DYNAMIT, we conducted an evaluation of its resource usage, focusing specifically on RAM and CPU consumption at idle. The evaluation was performed across four distinct scenarios: when the host machine is not running DYNAMIT, when the host is running only DYNAMIT's core system (0 profiles), when DYNAMIT is running with 5 honeypot profiles, and when running 30 honeypot profiles. Additionally, to contextualize DYNAMIT's resource footprint, we included a comparison with T-Pot running at Hive mode, as T-Pot is the implementation which DYNAMIT is built upon. The evaluation result are presented in Fig. 9.

The results indicate that both DYNAMIT and T-Pot exhibit a consistent CPU usage of approximately 5% at idle, regardless of the number of honeypot profiles deployed. In terms of memory consumption, however, there is a clear upward trend in RAM usage as DYNAMIT operates with an increasing number of profiles. Specifically, the difference in memory usage between the baseline system (with DYNAMIT not running) and the state in which only DYNAMIT's core system is active is approximately 5.3 GB, which can be considered the

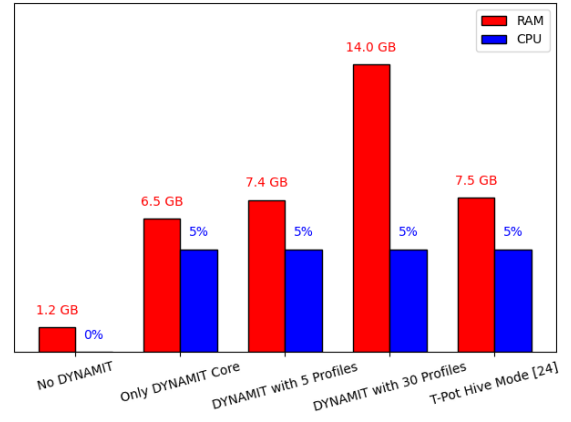


Fig. 9. Comparison of RAM and CPU usage when idle

baseline memory requirement for running DYNAMIT's core services.

When DYNAMIT is deployed with 5 honeypot profiles, the memory usage increases by 0.9 GB, indicating an average of approximately 180 MB of RAM per profile. In the case of 30 profiles, the memory usage increases by 7.5 GB, yielding an average of about 250 MB per profile. The variation in per-profile memory consumption may be attributed to factors such as container startup overhead and shared resource scaling. For comparison, T-Pot, when running in Hive mode, exhibits a memory footprint comparable to DYNAMIT operating with 5 profiles, suggesting that DYNAMIT introduces modest and scalable overhead in exchange for its dynamic orchestration capabilities.

D. Boot Time Evaluation

To clearly assess the system readiness after DYNAMIT is started, we conducted an evaluation upon the time required for the system to become fully operational, also known as boot time. The evaluation was performed in two scenarios: when it needs to run the honeypot profile creation process, and when it only runs pre-existing profiles. For each scenario, we measured the boot time in 3 cases: when host is only running DYNAMIT core components, when DYNAMIT is running 5 profiles, and running 30 profiles. As baseline comparison, we also measured the boot time for T-Pot in Hive mode. Boot time was determined by calculating the time difference between specific log messages recorded during the initialization process, marking the system's transition from startup to readiness.

In the first scenario, DYNAMIT is deployed within a production network containing the same number of hosts as the profile it needs to run, with the configuration parameter `HPOT_HOST_RATIO` is set to 1. The evaluation result are presented in Fig. 10

The results indicate that network scanning phase consistently completes in approximately 120 seconds, regardless of the amount of hosts present in the network. We can also clearly see that the more profile it runs, the longer it needs to run the orchestration process, and the longer the time taken to run the

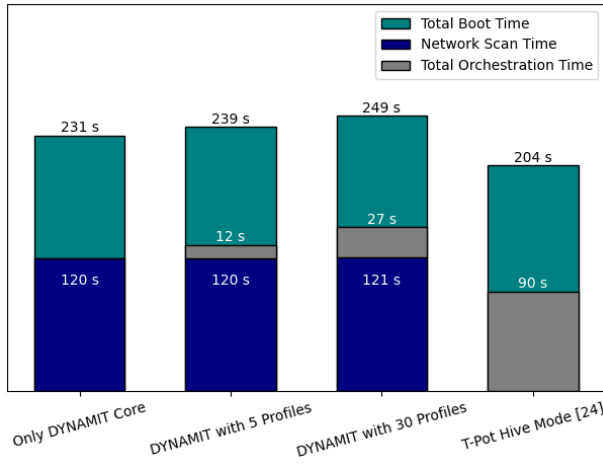


Fig. 10. Boot time comparison between DYNAMIT running different amount of profiles and T-Pot in Hive mode

whole boot process, from 231 seconds when running no profile to 249 seconds when running 30 profiles. This reflects the added overhead introduced by DYNAMIT's honeypot profile creation process. In comparison, T-Pot operating in Hive mode has a shorter boot time of approximately 204 seconds, highlighting that while DYNAMIT introduces adaptive deployment capabilities, it does so by sacrificing boot speed.

In the second scenario, DYNAMIT is configured to bypass the profile creation phase and instead run pre-existing honeypot profiles, ranging from 0 profiles (core components only) to 30 profiles. The results of this evaluation are presented in Fig. 11

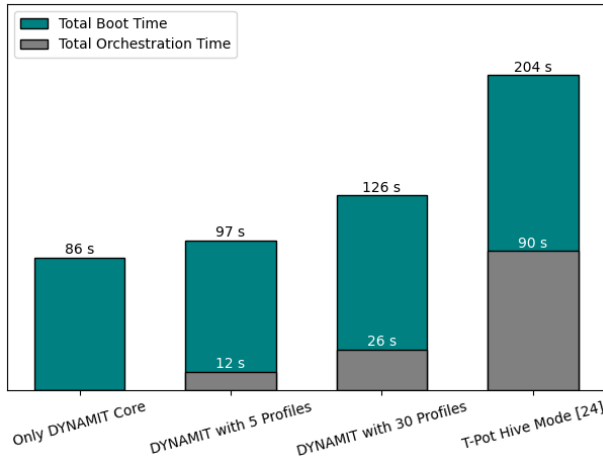


Fig. 11. Boot time comparison between DYNAMIT running different amount of profiles and T-Pot in Hive mode

By skipping the network scanning and profile creation phase, DYNAMIT reduces its boot time by approximately 123 to 143 seconds, depending on the number of profiles deployed. In this configuration, DYNAMIT demonstrates a notably faster startup time compared to T-Pot operating in Hive mode, highlighting the performance benefits of reusing pre-

generated honeypot profiles in scenarios where adaptability is not immediately required.

V. CONCLUSION

This study successfully designed and implemented DYNAMIT, a dynamic honeypot orchestration system capable of automatically deploying honeypot profiles that mimic the characteristics of hosts within a given network environment. The system integrates network scanning, clustering, and profile-based container deployment into a unified workflow. Overall, DYNAMIT demonstrates practical viability as a dynamic honeypot platform, offering automated deployment with minimal administrative overhead, while maintaining adaptability to network conditions.

Several directions can be pursued to further refine and extend the implementation and evaluation of the DYNAMIT system:

- 1) Large-scale deployment testing should be conducted on production networks with a high number of real hosts, in order to assess network clustering performance in real-world environments.
- 2) Red team and penetration testing exercises should be performed to evaluate how effectively DYNAMIT deceives and responds to adversarial behavior.
- 3) The system's emulation capabilities could be expanded to include additional deception techniques, such as service and OS fingerprint emulation, honeytokens, non-default credentials, and realistic web page imitation, to further increase believability and attacker engagement.
- 4) Future versions of DYNAMIT could be extended to support deployment across multiple VLANs, not just within a single LAN, in order to improve its flexibility and scalability in segmented network environments.
- 5) Alternative clustering algorithms, such as DBSCAN or agglomerative clustering, could be explored and compared against the current K-Medoids approach in terms of accuracy and robustness across varying network conditions.

REFERENCES

- [1] S. Morgan, "Cybercrime To Cost The World \$10.5 Trillion Annually By 2025," Nov. 2020. [Online]. Available: <https://cybersecurityventures.com/cybercrime-damage-costs-10-trillion-by-2025/>
- [2] "Cost of a data breach 2024," IBM. [Online]. Available: <https://www.ibm.com/reports/data-breach>
- [3] N. Provos and T. Holz, *Virtual Honeypots: From Botnet Tracking to Intrusion Detection*. Upper Saddle River, NJ: Munich: Addison-Wesley, 2010.
- [4] P. Lanka, K. Gupta, and C. Varol, "Intelligent Threat Detection—AI-Driven Analysis of Honeypot Data to Counter Cyber Threats," *Electronics*, vol. 13, no. 13, p. 2465, Jun. 2024. [Online]. Available: <https://www.mdpi.com/2079-9292/13/13/2465>
- [5] W. Fan, D. Fernández, and Z. Du, "Versatile virtual honeynet management framework," *IET Information Security*, vol. 11, no. 1, pp. 38–45, Jan. 2017. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1049/iet-ifs.2015.0256>
- [6] S. Srinivasa, J. M. Pedersen, and E. Vasilomanolakis, "Gotta catch 'em all: a Multistage Framework for honeypot fingerprinting," Sep. 2021, arXiv:2109.10652. [Online]. Available: <http://arxiv.org/abs/2109.10652>

- [7] S. Dasent, "Dynamic honeypots," *Wellington Faculty of Engineering Symposium*, Oct. 2023. [Online]. Available: <https://ojs.victoria.ac.nz/wfes/article/view/8364>
- [8] A. Javadpour, F. Ja'fari, T. Taleb, M. Shojafar, and C. Benzaïd, "A comprehensive survey on cyber deception techniques to improve honeypot performance," *Computers & Security*, vol. 140, p. 103792, May 2024. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0167404824000932>
- [9] Y. Zhang and Y. Shi, "Constructing dynamic honeypot using machine learning," in *Proceedings of the 8th International Conference on Cyber Security and Information Engineering*, ser. ICCSIE '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 116–120. [Online]. Available: <https://doi.org/10.1145/3617184.3618056>
- [10] D. Fraunholz, M. Zimmermann, and H. D. Schotten, "An Adaptive Honeypot Configuration, Deployment and Maintenance Strategy," Nov. 2021, arXiv:2111.03884. [Online]. Available: <http://arxiv.org/abs/2111.03884>
- [11] U. Bartwal, S. Mukhopadhyay, R. Negi, and S. Shukla, "Security Orchestration, Automation, and Response Engine for Deployment of Behavioural Honeypots," in *2022 IEEE Conference on Dependable and Secure Computing (DSC)*. Edinburgh, United Kingdom: IEEE, Jun. 2022, pp. 1–8. [Online]. Available: <https://ieeexplore.ieee.org/document/9888808/>
- [12] C. Song, Y. Park, K. Golani, Y. Kim, K. Bhatt, and K. Goswami, "Machine-learning based threat-aware system in software defined networks," in *2017 26th International Conference on Computer Communication and Networks (ICCCN)*, 2017, pp. 1–9.
- [13] E. Schubert and P. J. Rousseeuw, "Fast and eager k-medoids clustering: O(k) runtime improvement of the pam, clara, and clarans algorithms," *Information Systems*, vol. 101, p. 101804, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306437921000557>
- [14] E. Schubert and L. Lenssen, "Fast k-medoids clustering in rust and python," *Journal of Open Source Software*, vol. 7, no. 75, p. 4183, 2022. [Online]. Available: <https://doi.org/10.21105/joss.04183>
- [15] "2023 vulnerability statistics report, 8th edition," Edgescan, Tech. Rep., Mar. 2024. [Online]. Available: <https://www.edgescan.com/wp-content/uploads/2024/03/2023-Vulnerability-Statistics-Report.pdf>